

Smart Box System Architecture & Workflow Guide

This document serves as the comprehensive, end-to-end guide explaining how the entire Smart Box ecosystem functions. It details the architecture, the responsibilities of each component, and the step-by-step workflows that make the system tick.

1. System Overview

The Smart Box ecosystem is a tripartite system composed of custom hardware, a mobile application, and a cloud-based web dashboard. It enables users to locate, unlock, and utilize EV charging and 3-pin power sockets on a pay-per-use basis.

Core Components

1. ESP32 Hardware (Firmware in C++)

- The physical smart box acting as the endpoint.
- Controls physical hardware: Magnetic lock, EV charger relay (3.3kW), and 3-Pin socket relay.
- Collects telemetry: Monitors voltage, current, power, and energy consumption using dual PZEM-004T v3.0 sensors.
- User Interface: Features a 128x64 OLED display and a 4x4 matrix keypad via PCF8575 I2C expander.
- Interacts strictly with the Next.js API backend to pull commands and push state changes.

2. Mobile Application (Flutter / Dart)

- The primary interface for end-users and on-the-ground administrators.
- Features: Google Maps integration for locating boxes, Wallet system with Razorpay integration, QR code scanning, and live session monitoring.
- Contains a dedicated **Box Provisioning** flow for securely transferring credentials to newly deployed ESP32 hardware.

3. Admin Dashboard (Next.js / React)

- The central command center for system administrators.
- Features: User management, box management, remote overrides, tariff editing, and financial statistics.
- Hosts the **Serverless API Routes** (`/api/esp/*`) that the ESP32 hardware relies on for communication.

4. Database (Firebase / Firestore)

- The central source of truth. Connects the Mobile App, Admin Dashboard, and ESP32 Backend together using real-time sync.
-

2. The Provisioning Workflow (A to Z)

How a physical ESP32 box goes from being assembled in a workshop to being live on the cloud network.

1. **Box Creation:** The administrator logs into the **Next.js Web Dashboard** and clicks "Add Box". They assign the box an ID and an owner.
 2. **Secret Generation:** The dashboard securely generates a random, unique `deviceSecret` and a 6-digit `Registration ID`. The `deviceSecret` is saved to the Firestore `boxes` database.
 3. **App Verification:** The administrator hands the 6-digit Registration ID to the user deploying the box. The user opens the **Flutter Mobile App**, enters the ID, and the app fetches the secret key from the cloud.
 4. **Local Network Handshake:** The ESP32 is powered on for the first time. Because it lacks credentials, it creates a local Wi-Fi Hotspot (`SmartBox_Setup`). The user's phone connects to this hotspot.
 5. **Configuration Transmission:** The Flutter app sends an HTTP POST request to the ESP32 (`192.168.4.1/configure`) containing the local Wi-Fi SSID, Password, Box ID, and the unique `deviceSecret`.
 6. **Reboot & Connect:** The ESP32 saves these credentials to its non-volatile memory (EEPROM/Preferences) and reboots. It connects to the local Wi-Fi router and begins polling the cloud. The app updates the box status to `available` in Firestore.
-

3. Hardware Communication Workflow

The ESP32 does not listen for incoming connections from the internet (which would require complex port-forwarding). Instead, it **polls** the server.

1. **Fetching Commands (`GET /api/esp/next-command`):**
 - Every 5 seconds, the ESP32 sends an HTTP GET request to the Next.js backend.
 - It includes its `Box ID` and `deviceSecret` in the HTTP headers.
 - The backend securely authenticates the request against the database. If authorized, the backend checks the `commands` collection in Firestore for any pending actions (e.g., unlocking the door).
 - The backend translates these into hardware instructions and replies to the ESP32.
 2. **Executing Hardware Actions:**
 - The ESP32 parses the JSON reply. If a command dictates an unlock, it triggers the lock relay for 1 second. If a relay command dictates turning on the EV charger, it flips the GPIO pin high and resets the energy meter.
 3. **Telemetry & Acknowledgement (`POST /api/esp/ack`):**
 - Immediately after executing commands, the ESP32 reads the physical state of the lock, the state of the relays, and pulls the live electrical telemetry from the PZEM sensors.
 - It packages this into a JSON payload and POSTs it back to the backend.
 - The backend updates the Firestore `boxes` document with the live voltage/current/energy, marks the command as `completed`, and updates the `lastHeartbeat` timestamp (letting the system know the box is Online).
-

4. User Charging Workflow

How an end-user actually utilizes the system to charge their vehicle.

1. **Wallet Recharge:** The user adds funds to their digital wallet in the Flutter app using Razorpay.
2. **Initiating Session:** The user locates a box, scans its QR code (or selects it on the map), and taps "Start Session" for either the EV Charger or 3-Pin socket.
3. **Command Queue:** The Flutter app writes a **pending** command to the Firestore **commands** collection, requesting the relay to turn on.
4. **Activation:** Within 5 seconds, the ESP32 polls the backend, picks up the command, turns on the physical relay, and acknowledges it. The user's app sees the acknowledgment via Firestore real-time sync and starts a timer.
5. **Live Monitoring:** As the ESP32 continuously sends telemetry (**POST /api/esp/ack**), the backend updates Firestore, allowing the Flutter app to display live power draw and energy consumption to the user.
6. **Ending Session:** The user taps "End Session". A stop command is queued, the ESP32 turns off the relay, and the backend calculates the final cost based on the total **energy** consumed multiplied by the box's specific **tariff** rate. The cost is deducted from the user's wallet.